

التقرير الفني النهائي لمشروع الحوسبة السحابية والحاويات

نظام إدارة المهام باستخدام Docker و REST API

اسم المقرر: حوسبة سحابية

اسم المشروع: Task Management API

أعضاء الفريق:

١. [اسم عضو الفريق الأول]

٢. [اسم عضو الفريق الثاني]

٣. [اسم عضو الفريق الثالث]

٤. [اسم عضو الفريق الرابع]

الدكتور: [اسم الدكتور]

الجامعة: [اسم الجامعة]

العام الدراسي: ٢٠٢٦

جدول المحتويات

١	 الملخص التنفيذي (Executive Summary)	2
٢	 أهداف المشروع	2
٣	 نظرة عامة على النظام	2
٤	 معمارية النظام (System Architecture)	3
٥	 REST API و CRUD	3
٦	 قاعدة البيانات PostgreSQL	3
٧	 Virtualization vs Containerization	4
٨	 Dockerfile وتحسين الطبقات	4
٩	 Docker Compose	5
١٠	 Network Architecture	5
١١	 Nginx Reverse Proxy	5
١٢	 Persistence و Storage Architecture	5
١٣	 إثبات Persistence	5
١٤	 Security Practices	6
١٥	 Failure Mode Scenarios	6
١٦	 Testing Evidence	6
١٧	 Docker Hub / Registry & Versioning	6

١٨	Reproducible Deployment.....	7
١٩	GitHub و Git	7
٢٠	Team Contributions	7
٢١	خطوات تشغيل المشروع	7
٢٢	الخاتمة	7
٢٣	Verification Status.....	7

١ الملخص التنفيذي (Executive Summary)

يقدم هذا التقرير تفصيلاً فنياً لتصميم وبناء نظام إدارة المهام (Task Management API). يهدف المشروع إلى حل مشكلة عدم توافق بيئات التشغيل (Environment Inconsistency) وتعقيد النشر (Deployment Complexity) من خلال استخدام تقنية الحاويات (Containerization) ممثلة في Docker.

يتكون النظام من واجهة برمجة تطبيقات (REST API) مبرمجة باستخدام Node.js وإطار العمل Express، والتي تتواصل مع قاعدة بيانات علائقية (Relational Database) ممثلة في PostgreSQL. ولضمان الأمان وتوجيه الطلبات بشكل مركزي، تم استخدام وكيل عكسي (Reverse Proxy) بواسطة Nginx.

استُخدم Docker Compose لربط هذه المكونات معاً لتشغيلها كبيئة واحدة معزولة، وتمت محاكاة سيناريوهات الفشل (Failure Modes) وإثبات استمرارية البيانات (Persistence). وأخيراً، تم رفع صورة النظام (Docker Image) إلى السجل العام (Docker Hub) لضمان سهولة التوزيع والنشر.

٢ أهداف المشروع

يهدف هذا المشروع إلى تحقيق عدة مخرجات تعليمية وهندسية أساسية، تشمل:

- بناء REST API: تطبيق عمليات الإنشاء والقراءة والتحديث والحذف (CRUD).
- التشغيل المعزول (Containerization): عزل كل تقنية في حاوية (Container) مستقلة.
- استخدام Docker Compose: توحيد التشغيل للخدمات المتعددة.
- وكيل عكسي (Reverse Proxy): استخدام Nginx لحماية وتوجيه الطلبات.
- العزل الشبكي (Network Isolation): حماية الخدمات الخلفية من الوصول المباشر.
- تخزين مستمر (Persistent Storage): تخزين البيانات في وحدة تخزين مسماة (Named Volume).
- اختبار الفشل (Failure Mode Testing): التحقق من قدرة النظام على التعامل مع التوقف.
- إدارة الإصدارات (Versioning): رفع الصور إلى السجل (Docker Registry).
- التشغيل المتطابق (Reproducible Deployment): القدرة على تشغيل المشروع بسهولة وفي أي بيئة.

٣ نظرة عامة على النظام

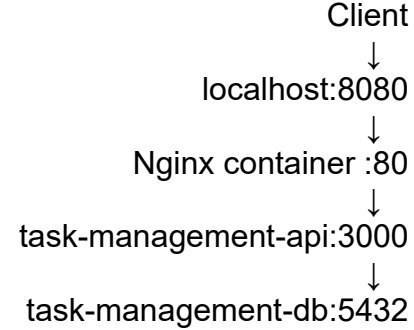
يبدأ مسار الطلب من العميل ويتدرج كالتالي:

PostgreSQL → Node.js / Express API → Nginx → Client

١. Nginx: يستقبل الطلب من خارج النظام ويوجهه للداخل.
٢. Node.js API: يستقبل الطلب الموجه، يحلل البيانات، وينفذ منطق العمل.
٣. PostgreSQL: ينفذ استعلام قاعدة البيانات لحفظ أو جلب المهام.

٤ معمارية النظام (System Architecture)

يعتمد المشروع على معمارية ثلاثية الطبقات (٣-Tier Architecture) معزولة كلياً. مسار الطلب الدقيق والمنافذ يعمل كالتالي:



- Nginx: يعمل على المنفذ ٨٠ داخل الحاوية. تم ربطه بمنفذ المضيف 8080 (Host) (أي ٨٠:٨٠٨٠) ليكون نقطة الدخول الخارجية الوحيدة.
- API: يعمل على المنفذ ٣٠٠٠ داخل شبكة Docker ولا يتم نشره للمضيف (Host). يقوم Nginx بتوجيه الطلبات إليه عبر العنوان الداخلي task-management-api:3000.
- PostgreSQL: يعمل على المنفذ ٥٤٣٢ داخل شبكة Docker، وهو معزول تماماً عن المضيف ويتصل به الـ API عبر العنوان task-management-db:5432.

٥ REST API وCRUD

واجهة برمجة التطبيقات (REST API) هي حلقة الوصل التي تستقبل طلبات (HTTP Requests) العميل وترد عليها (HTTP Responses). تم تطبيق عمليات CRUD الأساسية كالتالي:

العملية	HTTP Method	Endpoint
إنشاء	POST	api/tasks/
قراءة	GET	api/tasks/
تعديل	PUT	api/tasks/:id/
حذف	DELETE	api/tasks/:id/

٦ قاعدة البيانات PostgreSQL

تم اختيار PostgreSQL لقوتها كقاعدة بيانات علائقية. يتكون النظام من قاعدة بيانات (Database) تحت اسم taskdb تحتوي على جدول (Table) يسمى tasks. كل مهمة تمثل صفاً (Row) يتكون من أعمدة (Columns) مثل id (كمفتاح أساسي Primary Key)، و title، و status. يتم التواصل معها عبر أوامر SQL ويرتبط بها الـ API عبر اسم خدمة Docker الداخلي: task-management-db:5432 وليس localhost.

Virtualization vs Containerization ٧

لإثبات كفاءة معماريتنا، نوضح الفرق بين الأجهزة الافتراضية والحاويات:

الخاصية	Virtual Machine	Container
Guest OS	يتطلب نظام تشغيل كامل لكل آلة	لا يوجد، تشارك نواة المضيف (Host Kernel)
الموارد	ثقيلة وتستهلك RAM/CPU مقتطعة	خفيفة جداً تستهلك موارد حسب الحاجة
Startup Time	دقائق	أجزاء من الثانية (Milliseconds)
Isolation	عزل قوي (Hardware Level)	عزل على مستوى العمليات (Process Level)
Portability	صعبة النقل (ملفات بحجم جيجابايت)	سهلة النقل والتوزيع (ميجابايت)
Image	ملفات كبيرة (VHD/VMDK)	صور مبنية كطبقات خفيفة
Deployment	يتطلب إعدادات بنية تحتية معقدة	سريع باستخدام Dockerfile
Scalability	بطيء في التوسع	سريع جداً في إنشاء حاويات جديدة
		يعتبر Docker الخيار الأمثل لهذا المشروع لأنه يسمح بتشغيل الطبقات الثلاث على نفس الجهاز باستهلاك موارد ضئيل وبسرعة فائقة.

Dockerfile وتحسين الطبقات ٨

إليك ملف api/Dockerfile الفعلي المستخدم لبرمجة صورة الـ API:

```
FROM node:24-alpine
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm install --omit=dev
COPY . .
USER node
EXPOSE 3000
CMD ["npm", "start"]
```

- FROM: يعتمد على صورة مصغرة alpine لتخفيف الحجم والسرعة.
- WORKDIR: يحدد مجلد العمل داخل الحاوية.
- COPY package*.json ./: تم وضع هذا الأمر متبوعاً بـ npm install قبل أمر نسخ الكود المصدري COPY . . هذا يسمح بالاستفادة القصوى من التخزين المؤقت للطبقات (Layer Caching) لضمان عدم إعادة التنصيب إلا إذا تغيرت المكتبات فعلياً.
- npm install --omit=dev: يثبت حزم الإنتاج فقط لتقليل الحجم وتحسين الأمان.
- USER node: يحول المستخدم من root إلى node لتعزيز الأمان.
- EXPOSE 3000: يوثق أن المنفذ ٣٠٠٠ مستخدم داخلياً، ولكنه لا يفتحه على المضيف.
- CMD: يحدد الأمر الافتراضي لتشغيل الخدمة.

٩ Docker Compose

ملف `compose.yaml` هو القلب النابض للنظام، حيث يعرف الخدمات:

- `task-management-db`
- `task-management-api`
- `task-management-nginx`

استخدمنا `depends_on` مع `healthcheck` لضمان عدم عمل الـ `API` إلا بعد جاهزية قاعدة البيانات تماماً. وُضعت المتغيرات الأمنية في قسم `environment`، وتم ربط جميع الخدمات في شبكة `networks`. تم تخصيص `ports` لحاوية `nginx` فقط لإبقاء الـ `API` و `PostgreSQL` مخفيين تماماً عن المضيف الخارجي.

١٠ Network Architecture

يعتمد المشروع على شبكة فعلية من نوع `Bridge Network` تسمى `task-management-network`. توفر هذه الشبكة خدمة نظام أسماء النطاقات الداخلي (`Internal DNS`) واكتشاف الخدمات (`Service Discovery`). بفضل هذا، يمكن لحاوية الـ `API` التحدث مع قاعدة البيانات بمجرد استخدام اسم الخدمة `task-management-db`، ويتواصل `nginx` مع الـ `API` عبر `task-management-api` دون الحاجة لمعرفة عنوان الـ `IP` المتغير.

١١ Nginx Reverse Proxy

يعمل `nginx` كوكيل عكسي (`Reverse Proxy`)، ويحقق فوائد هامة: * نقطة دخول واحدة للمشروع بأكمله. * إخفاء واجهة الـ `API` عن المضيف تماماً. * عزل شبكي كامل (`Network Isolation`). * تنظيم وحماية حركة المرور.

١٢ Persistence و Storage Architecture

نظام ملفات الحاوية (`Container filesystem`) هو نظام مؤقت يُمحي بمجرد إيقاف الحاوية. لضمان بقاء البيانات، استخدمنا وحدة تخزين مسماة (`Named Volume`) تدعى `task-management-db-data` وتم ربطها بمسار `PostgreSQL: /var/lib/postgresql/data` هذا الإجراء يفصل دورة حياة البيانات عن دورة حياة الحاويات لضمان عدم ضياع المهام.

١٣ إثبات Persistence

تم إجراء اختبار فعلي لاستمرارية البيانات كالتالي: ١. إنشاء مهمة (`Task`) جديدة. ٢. إيقاف المشروع عبر `docker compose down`. ٣. إعادة تشغيل المشروع عبر `docker compose up -d`. ٤. استرجاع البيانات والتأكد من بقاء المهمة التي تم إنشاؤها مسبقاً.

تحذير: يرجى التفريق بين `docker compose down` الذي يوقف النظام بأمان، وبين أمر `docker compose down -v` الذي يقوم بحذف وحدات التخزين المسماة ويمسح جميع البيانات نهائياً.

١٤ Security Practices

ركزنا على الجوانب الأمنية من خلال تطبيق الإجراءات الفعلية التالية: * تشغيل API باستخدام USER node وليس root. * عدم نشر port 3000 الخاص بـ API على Host. * عدم نشر port 5432 الخاص بـ PostgreSQL على Host. * استخدام Nginx كنقطة الدخول الخارجية الوحيدة. * استخدام gitignore لمنع رفع الملفات المؤقتة والسرية. * استخدام dockerignore لتقليل حجم الصورة الناتجة. * عدم رفع secrets إلى مستودع GitHub. * تثبيت Production Dependencies فقط عبر --omit=dev.

١٥ Failure Mode Scenarios

تمت محاكاة انهيار النظام للتأكد من قدرته على التعامل مع الأخطاء (بناءً على الاختبارات الفعلية للمشروع):

التفسير	النتيجة	الحالة
Nginx لا يستطيع الوصول إلى API	Bad Gateway ٥٠٢	توقف API
API لا يستطيع تنفيذ استعلامات قاعدة البيانات	Internal Server Error ٥٠٠	توقف Database
نقطة الدخول الخارجية متوقفة	Connection Refused	توقف Nginx
API يعالج الطلب بشكل صحيح	Not Found ٤٠٤	Task ID غير موجود
API يرفض البيانات غير الصحيحة	Bad Request ٤٠٠	POST بيانات خاطئة

١٦ Testing Evidence

تم تنفيذ الاختبارات في بيئة المشروع الحقيقية وتم توثيقها كالتالي:

- **Automated GET Test**: يحتوي ملف `api/tests/test.js` على سكريبت يقوم بطلب GET آلياً ويتوقع رد ٢٠٠ OK (تم التحقق منه تشغيلياً - RUNTIME VERIFIED).
- **Manual Runtime Testing**: تم التحقق من عمليات POST و PUT و DELETE واختبارات سيناريوهات الفشل المختلفة (Failure Modes) يدوياً أثناء التشغيل للتأكد من الموثوقية (RUNTIME VERIFIED). لا يزعم المشروع وجود Full Automated CRUD Suite لأن التركيز كان على اختبارات التشغيل اليدوية للوظائف المتقدمة.

١٧ Docker Hub / Registry & Versioning

تم استخدام سجل حاويات (Docker Registry) في موقع Docker Hub لرفع وتوثيق الصورة الخاصة بـ API.

- المستودع (Repository): `hishamdiv/task-management-api`
 - الإصدار الثابت: `hishamdiv/task-management-api:v1.0.0`
 - أحدث إصدار: `hishamdiv/task-management-api:latest`
- يستخدم الوسم `v1.0.0` لتجميد إصدار مستقر لا يتغير (Immutable)، بينما يُستخدم الوسم `latest` كأشرطة توجيه لأحدث نسخة متوفرة لتسهيل عمليات السحب.

١٨ Reproducible Deployment

يمكن لأي مستخدم تشغيل المشروع بنجاح عبر سلسلة الأوامر التالية:

```
git clone https://github.com/hishamdiv/task-management-api.git
cd task-management-api
docker compose up -d
```

ثم للتحقق والتأكد:

```
docker compose ps
Invoke-RestMethod -Uri "http://localhost:8080/api/tasks" -Method GET
```

١٩ Git و GitHub

تم رفع المشروع إلى مستودع التحكم بالإصدار (Repository) على GitHub. تمت إدارة التغييرات عبر لقطات (Commits) على الفرع الأساسي المسمى main، ومن ثم دفعها إلى الخادم البعيد (Remote).

٢٠ Team Contributions

المسؤوليات	عضو الفريق
Backend Development & Containerization	[اسم عضو الفريق الأول]
Reverse Proxy & Security	[اسم عضو الفريق الثاني]
Database & Storage	[اسم عضو الفريق الثالث]
Testing & Documentation	[اسم عضو الفريق الرابع]

٢١ خطوات تشغيل المشروع

الأوامر الأساسية لتشغيل واختبار المشروع محلياً:

```
cd "C:\Users\NOMAN\Projects\task-management-api"
docker compose config
docker compose up -d --build
docker compose ps
```

٢٢ الخاتمة

أثبت هذا المشروع نجاحه في تقديم معمارية قوية ومرنة تعتمد على حاويات Docker، حيث تم بناء واجهة تطبيقات مستقرة ومعزولة شبكياً ومحمية بواسطة وكيل عكسي. مع ضمان استمرارية البيانات وإدارة الانهيئات بطريقة سليمة، يحقق المشروع المتطلبات الأساسية للنشر السحابي الحديث.

٢٣ Verification Status

الحالة	العنصر
RUNTIME VERIFIED	Docker Compose

العنصر	الحالة
API	RUNTIME VERIFIED
PostgreSQL	RUNTIME VERIFIED
Nginx	RUNTIME VERIFIED
Network Isolation	RUNTIME VERIFIED
Persistence	RUNTIME VERIFIED
CRUD	RUNTIME VERIFIED
Docker Hub	RUNTIME VERIFIED
Failure Modes	RUNTIME VERIFIED
Automated GET Test	RUNTIME VERIFIED
Full Automated CRUD Suite	NOT IMPLEMENTED / NOT CLAIMED